

Blatt 05: ANTLR, Visitor; Äquivalenzklassen & Grenzwerte, Mocking

ANTLR

Variante	Aufwand	Fehleranfällig
RegexHighlighter	hoch	sehr hoch
ScanningHighlighter	mittel	mittel
ANTLR Token Highlighter	niedrig	sehr gering

- ANTLR kennt die Grammatik (erkennt Keywords, Literale, Kommentare exakt)
- Regex kann z.B. "class" in "classification" highlighten, was fehlerhaft ist

<https://github.com/NikitaTsundora/Syntax-Highlighting>

Cycle Chronicles

Analyse der Äquivalenzklassen & Grenzwerte

Notizen an mich selbst (nicht Teil der Aufgabe):

Äquivalenzklassen sind Teilmengen, in denen Objekte zusammengefasst werden, die in einem bestimmten Bezug (der sogenannten Äquivalenzrelation) als gleichwertig oder ähnlich gelten. Eine Äquivalenzrelation teilt eine Gesamtmenge lückenlos in disjunkte (überschneidungsfreie) Klassen ein.

Die drei Grundregeln der Äquivalenz:

Damit Äquivalenzklassen gebildet werden können, muss die zugrundeliegende Beziehung drei Eigenschaften erfüllen:

1. Reflexivität: Jedes Element steht in der Beziehung zu sich selbst (z. B. „ist genauso schwer wie“ – ein Objekt ist immer so schwer wie es selbst).
2. Symmetrie: Wenn Objekt A in der Beziehung zu B steht, gilt das auch umgekehrt für B und A (z. B. „ist verwandt mit“).
3. Transitivität: Überträgt sich die Beziehung von A auf B und von B auf C, gilt sie auch direkt von A auf C

Funktionsweise der Grenzwert-Analyse

Die Methode baut in der Regel auf der Äquivalenzklassenbildung auf. Anstatt jeden einzelnen Wert zu testen, teilt man Eingaben in gültige und ungültige Bereiche ein und testet gezielt die Werte direkt an den Grenzen.

Aufgabe:

Fahrrad:

Äquivalenzklassen:

- ✓ **(true):** Alle Fahrrad-Typen außer E-Bike und Gravel-Bike
- **(false):** Das Fahrrad ist ein E-Bike
- **(false):** Das Fahrrad ist ein Gravel-Bike

Grenzwerte:

Es gibt keine Grenze, aber zwei Fahrrad Arten werden ausgeschlossen: E-Bike und Gravel-Bike

Offene Aufträge des Kunden:

Äquivalenzklassen:

- ✓ **(true):** Der Kunde hat aktuell 0 offene Aufträge.
- **(false):** Der Kunde hat bereits mindestens einen offenen Auftrag

Grenzwerte:

Die Grenze liegt beim Übergang von 0 zu 1 offenen Aufträgen.

Grenzwert 0: **(true)** – Der Kunde hat noch keinen Auftrag.

Grenzwert 1: **(false)** – Der Kunde hat bereits einen offenen Auftrag.

Gesamtanzahl offener Aufträge im Shop:

Äquivalenzklassen:

- ✓ **(true):** Es existieren aktuell 0 bis 4 offene Aufträge im System
- **(false):** Es existieren bereits 5 oder mehr offene Aufträge

Grenzwerte:

Die Grenze für die aktuelle Anzahl der Aufträge liegt bei 4, da maximal 5 erlaubt sind.

- ✓ **Grenzwert 0:** **(true)** – Warteschlange ist komplett leer.
- ✓ **Grenzwert 4:** **(true)** – Warteschlange hat noch genau einen Platz frei
- **Grenzwert 5:** **(false)** – Warteschlange ist bereits voll besetzt.

Erstellen Sie aus den ermittelten Äquivalenzklassen und Grenzwerten konkrete Testfälle. (Noch keine Implementierung!)

Pro Grenzwert werden meist drei spezifische Testwerte ermittelt:

- **Der Grenzwert selbst:** Der exakte Wert, an dem die Bedingung kippt.
- **Knapp unter dem Grenzwert:** Der letzte gültige Wert vor dem Fehlerbereich.
- **Knapp über dem Grenzwert:** Der erste ungültige Wert nach dem gültigen Bereich.

Testfälle zu Bike-Typ

Nr.	Bike-Typ	Kundenaufträge	Andere offene Aufträge	Erwartung
T1	Fixed gear bike	0	0	true
T2	E-Bike	0	0	false
T3	Gravel-Bike	0	0	false

Testfälle zu Kundenaufträgen

Nr.	Bike-Typ	Kundenaufträge	Andere offene Aufträge	Erwartung
T4	Fixed gear bike	0	1	true
T5	Road bike	1	0	false
T6	Single speed bike	2	3	false

Testfälle zu Gesamtaufträgen

Nr.	Bike-Typ	Kundenaufträge	Andere offene Aufträge	Erwartung
T7	Single speed bike	0	3	true
T8	Road bike	0	4	true
T9	Road bike	0	5	false
T10	Fixed gear bike	0	6	false

Mocking 1

Warum ist Mockito hier notwendig?

Da die Klasse Order nicht implementiert ist, kann man die Methode Shop#accept nicht testen.

Mockito löst dieses Problem, indem wir Order-Objekte simulieren („mocken“).

So wird die Methode Shop#accept getestet, ohne die unvollständige Order-Klasse zu benötigen.

https://github.com/NikitaTsundora/prog2_ybel_cyclechronicles